



TEMPLATE

A Comparative Analysis of Random Search and Particle Swarm Optimization on an LSTM Model for USD/IDR Exchange Rate Data

Aurisa Putri Hawin Yusima¹, Erlangga Sri Heryanto², Fatima Azzahra³, Muhammad Fauzan Nur Rasendriya⁴, Ghonia Choirinnajdhatul Muna⁵, Akbar Rizki^{6*}, Bagus Sartono⁷, Sachnaz Desta Oktarina⁸, Rahma Anisa⁹, Indra Mahib Zuhair¹⁰

^{1,2,3,4,5,6,7,8,9,10}Department of Statistics and Data Science, School of Data Science, Mathematics, and Informatics, IPB University, Indonesia

*Corresponding Author: E-mail address: akbar.ritzki@apps.ipb.ac.id

ARTICLE INFO

Abstract

Article history:

Received dd month, yyyy

Revised dd month, yyyy

Accepted dd month, yyyy

Published dd month, yyyy

Keywords:

Long short-term memory;
Particle swarm optimization;
Random Search; USD/IDR
exchange rate

Forecasting the volatile USD/IDR exchange rate is crucial for Indonesian macroeconomic stability, and this study evaluates a Long Short-Term Memory (LSTM) model for predicting univariate exchange rate fluctuations from historical data. Although LSTM can capture non-linear financial patterns better than traditional ARIMA models, its performance is highly sensitive to hyperparameter configurations, raising the question of whether Particle Swarm Optimization (PSO) can provide more effective tuning than Random Search. This study addresses the gap in sequential data validation for metaheuristic optimization by integrating PSO with sliding-window cross-validation for univariate USD/IDR forecasting. Using historical daily USD/IDR data, LSTM hyperparameters were optimized with 20 particles over 30 iterations and benchmarked against Random Search under 3-fold sliding-window cross-validation. The PSO-LSTM model outperformed the baseline, achieving a RMSE of 0.013 compared with an RMSE of 0.017. These findings indicate that metaheuristic hyperparameter optimization can improve LSTM performance for volatile financial time-series forecasting.

1. Introduction

The exchange rate of the Indonesian Rupiah against the U.S. Dollar (USD/IDR) is a crucial fundamental macroeconomic indicator reflecting Indonesia's economic stability (Amri et al., 2025). Exchange rate movements have a direct and significant impact on domestic inflation rates, international trade activity, and investment flows (Sen & Minghui, 2026). Given increasingly tight global economic integration, emerging markets such as Indonesia are vulnerable to external shocks, capital flows, and currency depreciation, which can tighten domestic financial conditions (Bagsic et al., 2025; Greenwood-Nimmo et al., 2025; Juhro & Azwar, 2021). Currency depreciation triggers a pass-through effect in which the prices of imported goods rise, ultimately driving inflation and threatening macroeconomic stability (Ali et al., 2022; Sumba et al., 2024). These fluctuations are essentially driven by complex interactions among various macroeconomic and global indicators, such as central bank interest rates (Zheng, 2025).

However, modeling all these variables multivariately is often highly complex, a univariate forecasting approach that extracts direct patterns from historical USD/IDR exchange rate data provides an efficient and highly relevant method to develop (Hu et al., 2021).

Given the complexity and nonlinear nature of the movements of volatile financial instruments, univariate exchange rate forecasting remains a significant challenge. Traditional econometric approaches such as the Autoregressive Integrated Moving Average (ARIMA) have limitations in modeling nonlinear relationships and long-term dependencies in financial time series data (Hosseinzadeh et al., 2025). As an alternative, Long Short-Term Memory (LSTM) has demonstrated strong capabilities in capturing complex temporal patterns through its internal memory mechanism (Fischer et al., 2024; Wang et al., 2025). However, LSTM performance is highly influenced by hyperparameter selection, such as input sequence length, number of neurons, number of hidden layers, learning rate, dropout rate, batch size, and number of epochs. Determining hyperparameters manually or using simple search methods such as Random Search often requires significant computational costs and does not necessarily yield an optimal configuration.

Given these challenges, this study proposes LSTM hyperparameter optimization using Particle Swarm Optimization (PSO) to obtain a model configuration capable of minimizing the prediction error for the USD/IDR exchange rate. In this study, PSO is used to iteratively evaluate various hyperparameter combinations and guide the search process based on the best solutions previously found (Hosseinzadeh et al., 2025). The effectiveness of PSO-LSTM is then compared with Random Search-LSTM using an equivalent hyperparameter search space and number of model evaluations, with Random Search serving as the baseline for evaluating adaptive optimization methods. Thus, this study aims to analyze PSO's ability to improve the performance of USD/IDR exchange rate predictions and assess its potential as an optimization alternative for forecasting nonlinear and dynamic financial time series.

2. Material and Methods

2.1. Data

This study uses secondary data in the form of daily USD/IDR exchange rates based on the Jakarta Interbank Spot Dollar Rate (JISDOR) published by Bank Indonesia, covering the period from January 2015 to December 2025, with a total of 2,664 observations on business days. Data for Saturdays, Sundays, and national holidays are not available because the interbank foreign exchange market does not operate on those days. JISDOR is the USD/IDR spot rate compiled based on interbank USD/IDR transactions in the domestic foreign exchange market and collected in real time through Bank Indonesia's Foreign Exchange Transaction Monitoring System against the Rupiah (SISMONTAVAR). This indicator was selected as the main variable in the study because it serves as a representative and widely used market reference price that is utilized by both monetary authorities and market participants. Consequently, it is considered capable of reliably and consistently reflecting movements in the rupiah's exchange rate against the U.S. dollar.

2.2. Long Short Term Memory (LSTM)

In recent years, LSTMs have been widely applied in the fields of finance and economics, such as in stock price forecasting by (Zeng et al., 2025), macroeconomic indicator forecasting by (Wang et al., 2025), and forecasting the exchange rate of the Indonesian Rupiah against the U.S. Dollar by (Amri et al., 2025). LSTM is an extension of the Recurrent Neural Network (RNN) designed to address the vanishing gradient problem in RNNs, thereby enabling it to capture long-term dependencies in sequential data.

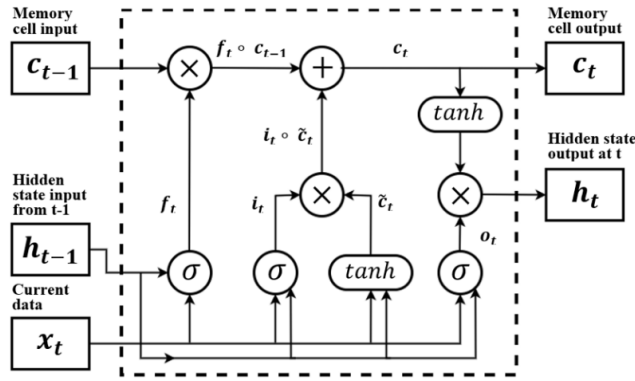


Figure 1. LSTM architecture

As shown in Figure 1, LSTM uses a gating mechanism consisting of an input gate, a forget gate, and an output gate to regulate the flow of information in the cell state. This mechanism allows the model to retain relevant information and disregard unnecessary information. These characteristics make LSTM effective in modeling time-series data, particularly macroeconomic data, which generally exhibits lag dependence (Khan et al., 2024).

The forget gate is used to determine which information from the previous cell state should be retained or discarded. This process is expressed in Equation (1):

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (1)$$

where W_f represents the forget gate weight matrix, h_{t-1} is the hidden state at the previous time step, x_t is the current input, b_f is the bias factor, and σ is the activation function.

Next, the input gate filters the new information to be stored in the cell state. This process consists of two stages: the formation of the information selection value via the sigmoid function and the formation of the cell state candidate via the hyperbolic tangent ($\tanh$) function, as stated in Equations (2) and (3):

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (2)$$

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (3)$$

where i_t is the input gate activation, while $\tilde{c}_t$ is the candidate cell state update.

The information from the forget gate and the input gate is then used to update the current cell state, as shown in Equation (4):

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{c}_t \quad (4)$$

where the operator $\odot$ denotes element-wise multiplication.

After updating the cell state, the output gate determines which part of the information will be passed on as the hidden state. The output gate value is calculated using Equation (5):

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (5)$$

the final hidden state is obtained using Equation (6):

$$h_t = o_t \odot \tanh(C_t) \quad (6)$$

To generate the predicted value, the hidden state from the previous step is passed to the output layer as shown in Equation (7):

$$\hat{y} = h_T W_y + b_y \quad (7)$$

where $\hat{y}$ is the predicted value, W_y is the weight in the output layer, and b_y is the bias.

The model training process is performed via forward propagation to generate predictions, after which the prediction error is calculated using a loss function. Next, the model parameters are updated using the Backpropagation Through Time (BPTT) algorithm to minimize this error.

Although LSTMs are effective at modeling sequential data, their performance is heavily influenced by the choice of hyperparameters, such as the number of neurons, learning rate, batch size, and number of epochs. Previous research has shown that hyperparameter optimization for LSTM can improve performance in terms of prediction accuracy, model stability, generalization ability, and robustness to data noise (Hosseinzadeh et al., 2025). Therefore, this study integrates LSTM with PSO to obtain an optimal hyperparameter configuration.

2.3. Random Search

Random Search is a hyperparameter tuning method that randomly searches for parameter combinations within a predetermined search space. Each hyperparameter has a range of values defined by a lower bound and an upper bound; the algorithm then randomly selects a number of combinations to evaluate based on model performance metrics. The hyperparameter combination that yields the best evaluation score is then designated as the optimal solution. This approach allows for the exploration of the search space without having to evaluate all possible parameter combinations (Nurdianto, 2024). This simple concept will serve as the baseline for comparing performance with the PSO optimization method using the same hyperparameter search space, as shown in Table 1.

2.4. Particle Swarm Optimization (PSO)

PSO is a population-based optimization algorithm inspired by the collective behavior of flocks of birds or schools of fish in search of food sources. This algorithm operates through cooperation and the exchange of information among particles in the population to find optimal solutions in a multidimensional search space (Kennedy & Eberhart, 1995).

Suppose there are N particles in a D -dimensional search space. Each particle has a position and velocity that are initialized randomly (Kennedy & Eberhart, 1995). The position of the i -th particle at the k -th iteration is given by Equation (8):

$$x_i^k = (x_{i1}^k, x_{i2}^k, \dots, x_{iD}^k) \quad (8)$$

while the velocity is defined in Equation (9):

$$v_i^k = (v_{i1}^k, v_{i2}^k, \dots, v_{iD}^k) \quad (9)$$

Each particle stores the best position ever achieved during the search process (personal best), as expressed in Equation (10):

$$pbest_i = (p_{i1}, p_{i2}, \dots, p_{iD}) \quad (10)$$

In addition, the entire population also stores the global best position, defined in Equation (11):

$$gbest = (p_{g1}, p_{g2}, \dots, p_{gD}) \quad (11)$$

At each iteration, a particle's velocity is updated based on the influence of its previous velocity, its tendency toward its individual best position, and its tendency toward the global best position. The update of a particle's velocity is expressed in Equation (12) (Kennedy & Eberhart, 1997).

$$v_{id}^k = wv_{id}^{k-1} + c_1r_1(pbest_{id} - x_{id}^{k-1}) + c_2r_2(gbest_d - x_{id}^{k-1}) \quad (12)$$

where w is the inertia factor that controls the particle's tendency to maintain its direction of motion, c_1 and c_2 are acceleration constants representing the influence of individual and social experience, respectively, and r_1 and r_2 are uniformly distributed random numbers used to increase the randomness of the search, with a value range between $[0, 1]$ (Zeng et al., 2025)

Next, the particle's position is updated using Equation (13):

$$x_{id}^k = x_{id}^{k-1} + v_{id}^k \quad (13)$$

this equation allows the particle to balance the exploration of the search space with the exploitation of the best solution already found. The inertia factor plays a role in expanding the search space, while the cognitive and social components help the particle move toward a more optimal solution region.

After the positions are updated, each particle is evaluated using the objective function to obtain a fitness value. This value is then used to update the individual's best position as shown in Equation (14) (Kennedy & Eberhart, 1995):

$$pbest_{i(k)} = pbest_{i(k-1)} \text{ if } f(x_{i(k)}) \geq f(pbest_{i(k-1)}), \text{ selainnya } x_{i(k)} \quad (14)$$

The global best position at the kth iteration is obtained via Equation (15):

$$gbest(k) = \arg \min i \{f(pbest_{i(k)})\} \quad (15)$$

this iterative process continues until the maximum number of iterations is reached or until the convergence criteria are met (Zeng et al., 2025). The global best solution obtained at the end of the optimization process is then selected as the optimal solution.

In this study, PSO was used to optimize the LSTM model's hyperparameters. Each particle represents a candidate hyperparameter combination consisting of the input sequence, the number of hidden layers, the number of neurons, the learning rate, the dropout rate, the batch size, and the number of epochs. The fitness value is calculated as the average RMSE from the sliding-window cross-validation results. Continuous hyperparameters are represented directly in the PSO search space, while discrete hyperparameters are rounded to the nearest integer after the particle's position is updated. The hyperparameter search space used in this study is presented in Table 1.

Table 1. Hyperparameter search space

Hyperparameter	Data Type	Value Range
Input sequence length	Integer	[10, 50]
Number of hidden layers	Integer	[1, 3]
Number of neurons per layer	Integer	[50, 300]
Learning rate	Continuous	[0,0001; 0,01]
Dropout rate	Continuous	[0,0; 0,5]
Batch size	Integer	[16, 128]
Number of epoch	Integer	[30, 100]

The learning rate and dropout rate hyperparameters are represented as continuous variables in the PSO search space. Meanwhile, the input sequence, number of hidden layers, number of neurons, batch size, and number of epochs are discrete variables; therefore, the particle position values in those dimensions are rounded to the nearest integer after the position update process. The rounded values are then used as hyperparameter configurations in the LSTM model training and evaluation processes.

2.5. Research Procedures

The research procedures in this study are as follows:

1. Data preparation. The steps involved include:
 - a. Check the completeness of the time series based on the observation dates to ensure there is no data loss due to recording errors.
 - b. Identify and remove duplicate data if there is more than one observation on the same date.
 - c. Interpolating data for weekends (Saturday–Sunday) and national holidays using the cubic spline interpolation method to produce a continuous, gap-free, and smooth daily time series (Yu, 2023). This interpolation allows the model to consistently capture temporal patterns across all calendar days. Each pair of points is interpolated using the following cubic polynomial:

$$S_k(x) = a_k x^3 + b_k x^2 + c_k x + d_k \quad (16)$$

where $k = 1, 2, \dots, n$ and n is the number of data points minus one, or in other words, the degree of the cubic polynomial..

2. Conduct data exploration to understand the characteristics of the data before the modeling process. The exploration steps include:
 - a. Visualizing the movement of the USD/IDR exchange rate using a line plot to identify trends and volatility.
 - b. Calculating descriptive statistics such as the mean, median, standard deviation, minimum value, and maximum value..
3. Splitting the dataset chronologically into three parts: 80% training data (3,213 periods), 10% validation data (402 periods), and 10% test data (402 periods).
4. Combining the training and validation data into a single tuning dataset, which is then normalized using a Min-Max Scaler so that all values fall within the range $[0, 1]$. The scaler parameters at this stage are calculated based on the combined training and validation data, while the test data is set aside and not included to avoid data leakage affecting the final evaluation results.
5. Applying 3-fold sliding-window cross-validation based on TimeSeriesSplit to the tuning data, with a maximum training window size of 500 observations. In each fold, the supervised dataset was generated using the sliding-window mechanism according to the input sequence length of the candidate hyperparameter being evaluated, and the fitness value was calculated as the average RMSE across all validation folds.

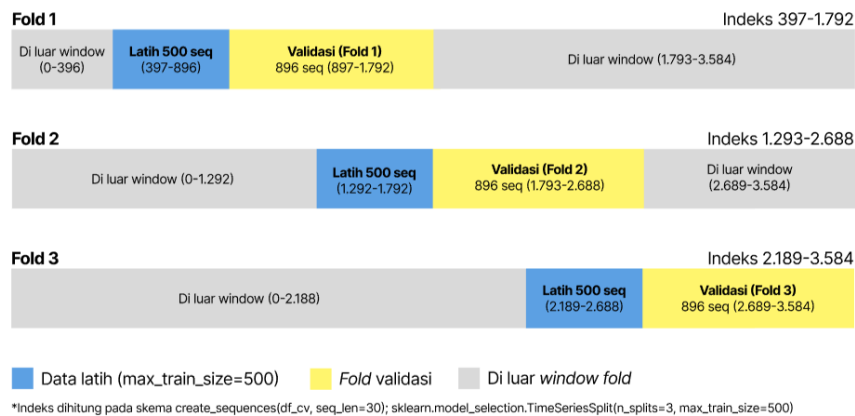


Figure 2. Illustration of sliding window cross-validation (sequence length = 30)

This scheme preserves the chronological order of the data and allows the model to be evaluated on newer data in stages. The lower the fitness value obtained, the better the quality of that hyperparameter combination.

6. Random Search was used as the baseline model by randomly sampling hyperparameter combinations (uniform random sampling) for 600 iterations from the same search space as PSO. The number of iterations was set so that the total number of model evaluations would be equivalent to that of PSO (20 particles $\times$ 30 iterations = 600), ensuring a fair comparison of the two methods in terms of the number of model evaluations. In each iteration, one hyperparameter combination is randomly selected and then evaluated using a sliding-window cross-validation scheme.
7. Performing hyperparameter optimization of the LSTM model using PSO with the parameter configurations presented in Table 3. Fitness evaluation was conducted using sliding-window cross-validation on the tuning data.

Table 3. PSO parameters

Parameter	Value
Numbers of particles	20
Maximum iterations	30
Inertia weight	0,7 (constant)
Cognitive coefficient	1,5

Social coefficient	1,5
Velocity bounds	[-2, 2]

The PSO parameters used in this study adopt the configuration reported by (Zeng et al., 2025), which demonstrated good performance in optimizing LSTM model hyperparameters. The use of the same configuration aims to maintain consistency with previous research while reducing the need for additional PSO parameter searches.

8. Building Random Search-LSTM and PSO-LSTM models using the optimal hyperparameter configurations obtained from each optimization method, with both models retrained on a combined training and validation dataset.
9. Conduct predictions on the test data by applying the two trained models to predict 402 test data observations that had not been used during either the optimization or training processes.
10. Evaluate performance using the following metrics:

- a. Mean Absolute Error

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (17)$$

- b. Root Mean Squared Error

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (18)$$

- c. Mean Absolute Percentage Error

$$MAPE = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \quad (19)$$

11. Comparing the performance of the Random Search-LSTM model with the PSO-LSTM model based on prediction results, all evaluation metrics used, and computational cost. This comparison aims to determine whether the PSO-based optimization approach can produce more optimal hyperparameter configurations than Random Search, with an equivalent number of model evaluations, so that any difference in performance purely reflects the contribution of the optimization mechanism used.

3. Result and Discussion

3.1. Data Preparation and Exploration

The research phase began with the preparation of daily USD/IDR exchange rate data (JISDOR) sourced from Bank Indonesia. The available data essentially covers only banking business days, so it does not include observations for Saturdays, Sundays, and national holidays. Before the data filling process, checks for the completeness of the time series and date duplicates were performed to ensure compliance with the assumptions of continuity and data quality; the results showed that there were no duplicate data points at all. Next, data gaps on non-business days were filled using cubic spline interpolation, which was chosen because it assumes continuity of the first and second derivatives in its local polynomial function. This mathematical approach is substantively capable of producing much smoother and more realistic intertemporal transition curves for foreign exchange market dynamics compared to linear interpolation, which tends to be rigid and jagged. This data reconstruction process produces a continuous daily time series, thereby increasing the number of observations from 2,664 working-day observations to 4,017 observations that cover all calendar days in their entirety during the period from January 2, 2015, to December 31, 2025. Representative samples of the interpolation results at the beginning and end of the study period are presented in Table 4.

Table 4. Cubic spline interpolation results

Date	USD/IDR Exchange Rate	Description
1/2/2015	12.474,00	Actual

1/3/2015	12.502,56	Cubic spline result
1/4/2015	12.539,04	Cubic spline result
1/5/2015	12.589,00	Actual
1/6/2015	12.658,00	Actual
1/7/2015	12.732,00	Actual
:	:	:
12/24/2025	16.767,00	Actual
12/25/2025	16.751,06	Cubic spline result
12/26/2025	16.751,98	Cubic spline result
12/27/2025	16.762,96	Cubic spline result
12/28/2025	16.777,23	Cubic spline result
12/29/2025	16.788,00	Actual
12/30/2025	16.782,00	Actual
12/31/2025	16.720,00	Actual

Source: Bank Indonesia (actual); this study (cubic spline results)

Based on the data structure presented in Table 4, it is evident that the cubic spline approach successfully estimates exchange rates on calendar holidays while maintaining the continuity of the trend from the surrounding actual data. Filling these time gaps is crucial in both spatiotemporal and time-series analyses to avoid structural bias resulting from information loss on non-trading days. Through this transformation, the hidden characteristics of daily volatility, which are typically overlooked in conventional time-series modeling, can now be captured more consistently and objectively.

After data preparation was completed, data exploration was conducted to understand the key statistical characteristics and temporal patterns of the USD/IDR exchange rate movements. Based on a total of 4,017 interpolated daily observations, the average exchange rate over the observation period stood at Rp14,526.94 per USD with a standard deviation of Rp1,049.37. Statistically, this high standard deviation reflects significant price fluctuations and a relatively high level of volatility risk in the domestic foreign exchange market over the past decade. This wide range of movement is underscored by the lowest recorded exchange rate of Rp12,444.00 and the highest rate, which exceeded Rp16,943.00, indicating a range of movement reaching Rp4,499.00. Meanwhile, the median value, which lies slightly below the mean, mathematically confirms that the data distribution is right-skewed, indicating that movements in the USD/IDR exchange rate were dominated by periods of extreme, short-term depreciation of the rupiah, which pulled the mean value upward. The time series visualization pattern along with a comprehensive breakdown of the data is presented in Figure 3.

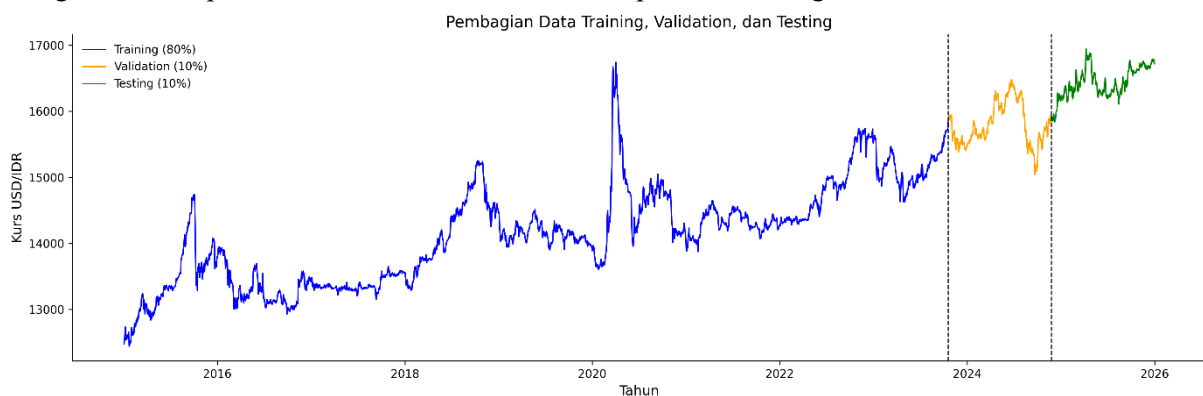


Figure 3. Plot of the daily USD/IDR exchange rate time series

Visually, the long-term depreciation trend of the rupiah against the U.S. dollar appears to form a pattern of consistent year-over-year increases. The exchange rate crept up from the Rp12,000–Rp13,000 range in early 2015 to settle at the upper end of the Rp15,000–Rp16,000 range toward the end of 2025. These fluctuations also reveal several clusters of high volatility, including the 2018–2019 period due to the Fed's monetary tightening, the massive spike in 2020 driven by risk sentiment surrounding the COVID-19 pandemic, and structural pressures toward the end of the period in 2024–2025 as the U.S. strengthened.

3.2. Hyperparameter LSTM with Random Search

The hyperparameter search method for the LSTM model in this research phase began by using Random Search as a baseline for comparison. This algorithm explores the search space in a purely stochastic manner by selecting 600 random combinations from a uniform distribution across each hyperparameter dimension. Each combination is then evaluated using a 3-fold sliding window cross-validation scheme on the training dataset to ensure the parameters' robustness against structural shifts over time. Fitness scores were calculated based on the average RMSE across the three validation folds, with the combination having the lowest absolute error value identified as the optimal configuration. Details of the best hyperparameter configuration obtained through the Random Search approach, along with its fitness score, are presented in Table 5.

Table 5. *Optimal Hyperparameters for Random Search-LSTM*

Hyperparameter	Optimal Value
Input sequence length	25
Number of hidden layers	1
Number of neurons per layer	94
Learning rate	0,005
Dropout rate	0,06
Batch size	34
Number of epochs	93
Fitness (average RMSE CV)	0,017

The optimal configuration from the Random Search results in Table 5 shows that the LSTM model requires an input sequence spanning 25 days, which is essentially equivalent to the estimated number of active trading days for foreign exchange transactions within a calendar month. The resulting network architecture is relatively parsimonious (simple), relying on only a single hidden layer with 94 neurons, which analytically indicates that the model limits the depth of the structure to prevent overfitting on financial data that is prone to noise. This algorithm employs a fairly aggressive learning rate of 0.005 combined with a small batch size, allowing weight updates during training to occur rapidly and dynamically in each epoch. On the other hand, the very low dropout rate selected indicates that the regularization mechanism is virtually disabled, forcing the model to rely entirely on the capacity of those 94 neurons to generalize data patterns. Overall, this independent combination was empirically able to produce a fitness value of 0.017 on the normalized data set. The historical trajectory of the search for this optimal combination over 600 evaluation iterations can be observed in the convergence curve in Figure 4.

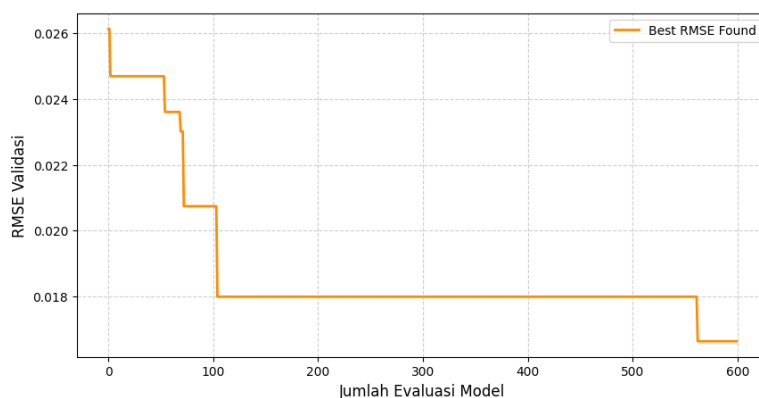


Figure 4. Random Search Convergence Curve

The convergence curve in Figure 4 illustrates the dynamics of the Random Search process, characterized by a discrete step-function pattern. In the early phase of the search, specifically during the first 100 evaluations, the best fitness value decreases at a fairly constant rate, gradually shifting from 0.026 to 0.018 as the probability of finding a favorable parameter space remains high. However, after the 100th evaluation, the curve experiences partial stagnation, marked by a flat line over an extended period. This phenomenon confirms a fundamental operational weakness of Random Search, namely its lack of computational memory, in which each point is sampled independently without considering

information from previous evaluations. As a result, improvements in model quality become purely a matter of chance (trial and error), as evidenced by a single additional decrease in RMSE to 0.017 as the evaluation approached the 600th cycle. Although iteratively inefficient in precisely exploiting the search space, this distribution of 600 random evaluations successfully establishes a good baseline RMSE estimate that is free from the researcher's initial subjective bias.

3.3. LSTM Hyperparameters with PSO

PSO optimizes LSTM hyperparameters through 30 iterations with 20 particles, resulting in a total of 600 model evaluations equivalent to Random Search. Each particle represents a candidate hyperparameter combination and updates its position iteratively based on the personal best and global best. Table 6 presents the optimal hyperparameter configurations obtained by PSO-LSTM along with their best fitness values.

Table 6. Optimal PSO-LSTM Hyperparameters

Hyperparameter	Optimal Value
Input sequence length	29
Number of hidden layers	1
Number of neurons per layer	151
Learning rate	0,008
Dropout rate	0,094
Batch size	19
Number of epochs	87
Fitness (average CV RMSE)	0,013

The optimal PSO-LSTM configuration shows several differences compared to Random Search-LSTM. The selected input sequence length is 29 days, slightly longer than 25 days, indicating that PSO identified longer temporal dependencies in the USD/IDR exchange rate data. The resulting architecture still consists of a single hidden layer with 151 neurons, thereby providing nearly 60% greater representational capacity. The learning rate of 0.008 is also slightly higher, accompanied by a dropout rate of 0.094, indicating more active regularization to reduce the risk of overfitting in the more complex architecture. Additionally, the small batch size allows for more frequent gradient updates, supporting smoother convergence. This combination of hyperparameters yields a fitness value of 0.013, which is 23.5% lower than that of Random Search-LSTM, indicating that PSO is capable of finding a more optimal configuration within the same search space.

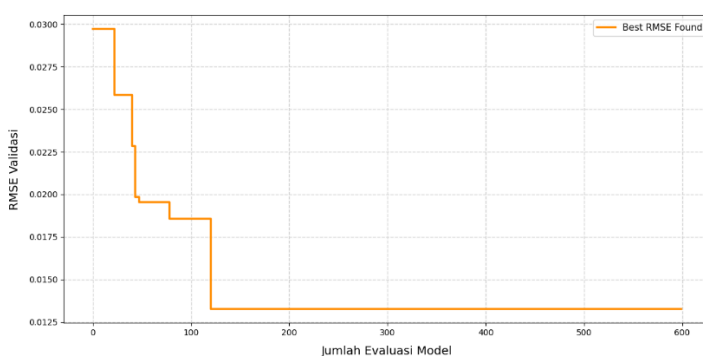


Figure 5. PSO convergence curve

The PSO convergence curve in Figure 5 exhibits characteristics that differ from those of Random Search. During the initial iterations, the global best fitness value decreases sharply as the particles extensively explore the search space. This rapid decline reflects the exploration phase of PSO, during which particle diversity remains high. As the optimization progresses, the rate of improvement gradually slows while remaining consistent, indicating a transition from exploration to exploitation around a promising solution region. From iterations 7 to 30, the curve becomes nearly flat, indicating that the swarm has converged to the global best solution. This structured convergence pattern differs

fundamentally from that of Random Search, as PSO actively guides all particles toward more promising regions of the search space by utilizing the collective information of the population. Consequently, PSO is able to achieve lower fitness values despite using the same number of evaluations.

3.4. Model Evaluation

Both models were retrained using a combination of training and validation data with their respective optimal hyperparameter configurations, then evaluated on test data that had not been seen during the optimization process. Table 7 presents a comprehensive comparison of the MAE, RMSE, and MAPE evaluation metrics on the validation and test data, along with the cross-validation runtime for each model.

Table 7. Comparison of evaluation metrics for Random Search-LSTM and PSO-LSTM

Model	Data	MAE	RMSE	MAPE	Runtime (CV)
Random Search-LSTM	Validation	67,79	88,77	0,454	
PSO-LSTM		56,21	75,34	0,376	
Selisih		11,58	13,43	0.078	
Random Search-LSTM	Test	54,62	63,09	0,332	5 hours 8 minutes
PSO-LSTM		37,49	46,96	0,227	4 hours 49 minutes
Difference		17,13	16,13	0,105	19 minutes

On the validation data, PSO-LSTM consistently outperformed Random Search-LSTM across all evaluation metrics. The MAE of 56.21 is 17.1% lower than that of Random Search-LSTM (67.79), corresponding to a reduction in the average absolute error of Rp11.58 per USD. Similarly, the RMSE decreased by 15.1%, from 88.77 to 75.34, indicating a reduction in larger prediction errors. The MAPE also declined from 0.454% to 0.376%, representing a relative error reduction of 0.078 percentage points. These results suggest that the PSO-optimized hyperparameter configuration provides better generalization on validation data that were not used during the training process.

This advantage becomes even more pronounced on the test data. The MAE decreased by 31.4%, from 54.62 to 37.49, while the RMSE was reduced by 25.6%, from 63.09 to 46.96. Likewise, the MAPE declined from 0.332% to 0.227%, indicating a smaller average prediction error relative to the actual exchange rate values. The greater improvement observed on the test data than on the validation data suggests that the hyperparameter configuration obtained through PSO optimization is more robust in predicting previously unseen data..

4. Conclusion

The application of PSO to optimize the LSTM model's hyperparameters using sliding-window cross-validation has significantly improved the accuracy of USD/IDR exchange rate forecasting, achieving an RMSE of 0.013 and outperforming the Random Search baseline. This approach provides a robust forecasting tool for policymakers and investors to anticipate currency depreciation risks while demonstrating the effectiveness of metaheuristic algorithms in navigating complex search spaces efficiently. Although the model exhibits strong internal validity, the univariate design adopted to isolate the effect of PSO optimization does not account for the influence of exogenous macroeconomic variables, limiting its responsiveness to sudden market shocks. In addition, the hyperparameter optimization process remains computationally intensive. Future research should extend this framework to a multivariate LSTM-PSO model that incorporates external macroeconomic indicators. The adoption of parallel computing or hybrid metaheuristic algorithms is also recommended to reduce computational costs and enable a broader exploration of the hyperparameter search space..

Ethics approval

Not required.

Acknowledgments

The author gratefully acknowledges the Department of Statistics, IPB University, for the support and facilities provided throughout this research. The author also thanks Bank Indonesia for providing and publishing the data used in this study.

Competing interest

All authors declare that there are no conflicts of interest in this study.

Funding

This study did not receive funding from external sources.

Underlying data

Derived data supporting the findings of this study are available from the corresponding author upon request.

Credit Authorship

Aurisa Putri Hawin Yusima: Conceptualization, Data Analysis, Writing—original draft, Visualization. **Erlangga Sri Heryanto:** Conceptualization, Methodology, Writing—original draft. **Fatima Azzahra:** Conceptualization, Data Collection, Methodology, Writing—original draft, Editing. **Muhammad Fauzan Nur Rasendriya:** Conceptualization, Writing—original draft. **Ghonia Choirinnajdhatul Muna:** Conceptualization, Writing—original draft. **Akbar Rizki:** Manuscript Review, Research Advisor. **Indra Mahib:** Research Advisor.

References

- [1] N. Ali, I. U. Hassan, and A. Wahab, "The Mediating Role of Inflation in the Relationship between Currency Depreciation and Economic Growth," *Global Social Sciences Review*, vol. VII, no. I, pp. 417–427, 2022, doi: 10.31703/gssr.2022(vii-i).38.
- [2] I. F. Amri, N. Yunanita, F. A. Lestari, and O. R. Dhani, "Rupiah exchange rate prediction against the US Dollar using a deep neural network with a multi-output sliding window approach," *MethodsX*, vol. 15, 2025, Art. no. 103692, doi: 10.1016/j.mex.2025.103692.
- [3] M. M. S. Amrir, Y. M. Ayid, A. M. Elshewey, and Y. Fouad, "A hybrid LSTM-GRU framework for lung cancer classification using GWO-WOA algorithm for hyperparameter tuning and BPSO for feature selection," *Scientific Reports*, vol. 16, no. 1, 2026, doi: 10.1038/s41598-026-39020-6.
- [4] C. Bagsic, V. Bayangos, R. Moreno, and H. Parcon-Santos, "The impact of currency depreciation and foreign exchange positions on bank lending: Evidence from an emerging market," *Emerging Markets Review*, vol. 66, 2025, Art. no. 101279, doi: 10.1016/j.ememar.2025.101279.

- [5] T. Fischer, M. Sterling, and S. Lessmann, "FX-spot predictions with state-of-the-art transformer and time embeddings," *Expert Systems with Applications*, vol. 249, 2024, Art. no. 123538, doi: 10.1016/j.eswa.2024.123538.
- [6] M. Greenwood-Nimmo, D. Steenkamp, and R. van Jaarsveld, "Risk and return spillovers among developed and emerging market currencies," *Journal of International Financial Markets, Institutions and Money*, vol. 98, 2025, Art. no. 102086, doi: 10.1016/j.intfin.2024.102086.
- [7] M. Hosseinzadeh, J. Tanveer, A. M. Rahmani, F. S. Gharehchopogh, N. M. Yusof, P. Khoshvaght, Z. Liu, T. Porntaveetus, and S. W. Lee, "A Comprehensive Overview of PSO-LSTM Approaches: Applications, Analytical Insights, and Future Opportunities," *Archives of Computational Methods in Engineering*, 2025, doi: 10.1007/s11831-025-10445-y.
- [8] Z. Hu, Y. Zhao, and M. Khushi, "A survey of forex and stock price prediction using deep learning," *Applied System Innovation*, vol. 4, no. 1, pp. 1–30, 2021, doi: 10.3390/asi4010009.
- [9] S. M. Juhro and P. Azwar, "FX Intervention Strategy and Exchange Rate Stability in Indonesia," *Bank Indonesia Working Paper*, 2021.
- [10] J. Kennedy and R. Eberhart, "Particle Swarm Optimization," in *Proceedings of the IEEE International Conference on Neural Networks (ICNN)*, 1995, pp. 1942–1948, doi: 10.1109/ICNN.1995.488968.
- [11] J. Kennedy and R. Eberhart, "A Discrete Binary Version of the Particle Swarm Algorithm," in *Proceedings of the 1997 IEEE International Conference on Systems, Man, and Cybernetics: Computational Cybernetics and Simulation*, 1997, pp. 4104–4108, doi: 10.1109/ICSMC.1997.637339.
- [12] M. A. Khan, H. Ali, H. Shabbir, F. Noor, and M. D. Majid, "Impact of Macroeconomic Indicators on Stock Market Predictions: A Cross Country Analysis," *Journal of Computing and Biomedical Informatics*, vol. 8, no. 1, 2024, doi: 10.56979/801/2024.
- [13] H. Nurdianto, "Prediksi Jumlah Hotspot di Kalimantan dengan Metode Regresi Proses Gaussian Berdasarkan Indikator Iklim," *Skripsi, Departemen Matematika, Fakultas Matematika dan Ilmu Pengetahuan Alam, Institut Pertanian Bogor, Bogor, Indonesia*, 2024.
- [14] B. R. Sen and J. Minghui, "Examining economic growth: Developed vs. developing nations in the face of exchange rate volatility," *Global Economics Research*, vol. 2, no. 1, Art. no. 100031, 2026, doi: 10.1016/j.ecores.2026.100031.
- [15] J. O. Sumba, K. O. Nyabuto, and P. J. Mugambi, "Exchange rate and inflation dynamics in Kenya: Does the threshold level matter?" *Heliyon*, vol. 10, no. 15, 2024, doi: 10.1016/j.heliyon.2024.e35726.
- [16] L. Wang, A. Hu, and X. Wu, "Prediction and Analysis of Macroeconomic Indicator from the Perspective of Deep Learning," in *Proceedings of the 2025 2nd International Conference on Image Processing, Intelligent Control and Computer Engineering (IPICE 2025)*, 2025, pp. 408–413, doi: 10.1145/3768184.3768256.
- [17] X. Yu, "Application Research of Spline Interpolation and ARIMA in the Field of Stock Market Forecasting," *arXiv preprint arXiv:2311.10759*, 2023. doi: 10.48550/arXiv.2311.10759.
- [18] X. Zeng, C. Liang, Q. Yang, F. Wang, and J. Cai, "Enhancing stock index prediction: A hybrid LSTM-PSO model for improved forecasting accuracy," *PLoS ONE*, vol. 20, no. 1, 2025, doi: 10.1371/journal.pone.0310296.

- [19] J. Zheng, "Exchange Rate Forecasts Based on Influencing Factors," 2025, doi: 10.54254/2754-1169/176/2025.22101.
- [20] Bank Indonesia, "JISDOR (Jakarta Interbank Spot Dollar Rate)," Bank Indonesia. [Online]. Available: <https://www.bi.go.id/id/statistik/informasi-kurs/jisdor/default.aspx>. [Accessed: Jul. 3, 2026].